In [ ]:
```python
import os
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from typing import List
```

In [ ]:
```python
torch.manual_seed(0)
np.random.seed(0)
```

In [ ]:
```python
class ResNetEncd(nn.Module):
    def __init__(self, base="resnet18", proj_dim=128):
        super().__init__()
        # Pretrained = True to extract powerful offline representations without
        import warnings
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            # Handle newer PyTorch API where pretrained is deprecated
            try:
                backbone = getattr(torchvision.models, base)(weights=torchvision
            except AttributeError:
                backbone = getattr(torchvision.models, base)(pretrained=True)

        self.backbone = nn.Sequential(*list(backbone.children())[:-1])
        feat_dim = backbone.fc.in_features
        self.projection = nn.Sequential(
            nn.Linear(feat_dim, feat_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feat_dim, proj_dim)
        )

    def forward(self, x, return_projection: bool = True):
        feat = self.backbone(x)
        feat = torch.flatten(feat, 1)
        if return_projection:
            proj = self.projection(feat)
            return feat, proj
        return feat

    def represent(self, x):
        return self.forward(x, return_projection=False)


def compute_typicality(features: np.ndarray, k: int = 20) -> np.ndarray:
    nbrs = NearestNeighbors(n_neighbors=min(k + 1, len(features)), algorithm='au
    distances, _ = nbrs.kneighbors(features)  # distances: [N, k+1]
    distances = distances[:, 1:]
    avg_sq_dist = (distances ** 2).mean(axis=1)
    typicality = 1.0 / (avg_sq_dist + 1e-12)
    return typicality

def typiclust_selection(
    features: np.ndarray,
    initial_labeled: List[int],
```

```
    budget_total: int,
    batch_size_per_round: int,
    k_typicality: int = 20,
    random_state: int = 0,
) -> List[int]:
    N = features.shape[0]
    rng = np.random.RandomState(random_state)

    labeled = set(initial_labeled)
    unlabeled = set(range(N)) - labeled

    typicality = compute_typicality(features, k=k_typicality)

    while len(labeled) < budget_total:
        current_L = list(labeled)
        current_U = list(unlabeled)

        n_clusters = len(current_L) + batch_size_per_round
        n_clusters = min(n_clusters, len(current_L) + len(current_U))  # safety

        kmeans = KMeans(n_clusters=n_clusters, random_state=rng.randint(1e9))
        cluster_labels = kmeans.fit_predict(features)

        cluster_has_label = np.zeros(n_clusters, dtype=bool)
        for idx in current_L:
            c = cluster_labels[idx]
            cluster_has_label[c] = True

        uncovered_clusters = [c for c in range(n_clusters) if not cluster_has_la

        cluster_sizes = {c: np.sum(cluster_labels == c) for c in uncovered_clust
        sorted_clusters = sorted(uncovered_clusters, key=lambda c: -cluster_size

        new_indices = []
        for c in sorted_clusters:
            if len(labeled) + len(new_indices) >= budget_total:
                break
            cluster_idx = np.where(cluster_labels == c)[0]
            cluster_idx = [i for i in cluster_idx if i in unlabeled]
            if not cluster_idx:
                continue
            cluster_typ = typicality[cluster_idx]
            best_local = cluster_idx[int(np.argmax(cluster_typ))]
            new_indices.append(best_local)

        for idx in new_indices:
            labeled.add(idx)
            unlabeled.remove(idx)

        if not new_indices:
            break

    return sorted(labeled)
```

In [ ]:

```
def extract_features(
    encoder: ResNetEncd,
    dataset: torchvision.datasets.CIFAR10,
    device: torch.device,
    batch_size: int = 256
) -> np.ndarray:
```

```python
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=4
    )

    encoder.eval()
    all_features = []

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            features = encoder.represent(x)
            all_features.append(features.cpu().numpy())

    features = np.concatenate(all_features, axis=0)
    return features


def run_pipeline_unsupervised(
    data_root: str = "./data",
    budget_total: int = 100,
    batch_size_per_round: int = 10,
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])

    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    # Use untrained representations OR imageNet representations.
    encoder = ResNetEncd(base="resnet18", proj_dim=128).to(device)

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    labeled_indices = typiclust_selection(
        features=features,
        initial_labeled=[],
        budget_total=budget_total,
        batch_size_per_round=batch_size_per_round,
        k_typicality=20,
        random_state=0,
    )
```

```
    os.makedirs("unsupervised_budget_results", exist_ok=True)
    np.save("unsupervised_budget_results/typiclust_unsupervised.npy", np.array(l
    print(f"Selected {len(labeled_indices)} labeled points (unsupervised)")

    return labeled_indices
```

In [ ]:

```
def generate_and_save_typiclust_selections(
        budgets=[10, 20, 40, 80],
        batch_size_per_round=10,
        data_root="./data"
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])

    base_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    # Use untrained representations OR imageNet representations.
    encoder = ResNetEncd(base="resnet18", proj_dim=128).to(device)

    features = extract_features(
        encoder=encoder,
        dataset=base_dataset,
        device=device,
        batch_size=256
    )

    os.makedirs("unsupervised_budget_results", exist_ok=True)

    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=batch_size_per_round,
            k_typicality=20,
            random_state=0,
        )

        np.save(f"unsupervised_budget_results/typiclust_B{B}.npy", np.array(labe
        print(f"Saved TypiClust selection for B={B} → unsupervised_budget_result
```

In [ ]:

```
if __name__ == '__main__':
    # run_pipeline_unsupervised()
    generate_and_save_typiclust_selections()
```